

Université Hassan II de Casablanca

Faculté des Sciences Ben M'Sik

Master d'Intelligence Artificielle

Module MLOps & DataOps

Prédiction du stress hydrique des cultures

**Conception et mise en œuvre
d'un pipeline DataOps/MLOps**

Réalisé par :

- Étudiant 1 : Anass Erfoudi
- Étudiant 2 : Jounaidi Oussama
- Étudiant 3 : Ismail Mousdik
- Étudiant 4 : El Marikh Rihab
- Étudiant 5 : Elbejjaji Kawtar
- Étudiant 6 : Abdessalam Safadi
- Étudiant 7 : Hamza Ennisal
- Étudiant 8 : Khaid Issil

Encadré par : Pr. Mohammed AIT DAOUD

Année universitaire : 2025–2026

Remerciements

Nous adressons nos remerciements à Pr. Mohammed AIT DAOUD pour le cadre pédagogique du module MLOps & DataOps. Nous remercions également la Faculté des Sciences Ben M'Sik et le Master d'Intelligence Artificielle de l'Université Hassan II de Casablanca. Ce travail mobilise des données et outils ouverts dont la documentation a rendu possible une démarche reproductible. Les choix scientifiques, les interprétations et les limites présentés dans ce rapport restent sous la responsabilité de l'équipe.

Résumé

Ce rapport présente la conception et l'implémentation d'un système DataOps/MLOps de prédiction du stress hydrique modélisé du blé sur trois sites marocains : Meknès, Settat et Marrakech. Les séries quotidiennes NASA POWER couvrent janvier 2015 à juin 2024, soit 10 407 observations. Un bilan hydrique à paramètres de sol hypothétiques fournit la cible du lendemain, comprise entre zéro et un. Le modèle utilise trente jours d'historique et vingt-cinq variables explicatives ; il n'est pas entraîné sur des mesures de stress au champ.

Le chemin DataOps associe dlt, DuckDB, dbt, un contrat de données Python/SQL et Dagster. Il reproduit les sorties du pipeline initial tout en isolant ses dépendances et ses résultats. La chaîne MLOps comprend une validation chronologique, quatre modèles, le suivi MLflow, un registre candidat/production, FastAPI, une interface Streamlit, Docker Compose, une CI GitHub Actions et des rapports de monitoring.

HistGradientBoosting est sélectionné par la validation. Sur 1 074 observations de test, la RMSE vaut 0,188591, la MAE 0,140200 et le R^2 0,625080. Une persistance fondée sur l'état simulé complet obtient une erreur plus faible. Les résultats démontrent une chaîne reproductible et l'approximation d'un proxy ; ils ne valident pas des décisions d'irrigation. Le rapport distingue les preuves techniques, les limites scientifiques et la reconstruction pédagogique des phases Agile.

Mots-clés : stress hydrique ; blé ; NASA POWER ; DataOps ; MLOps ; dlt ; DuckDB ; dbt ; Dagster ; MLflow ; validation chronologique.

Abstract

This report describes an end-to-end DataOps/MLOps system for next-day prediction of a modeled wheat water-stress index at three Moroccan sites: Meknes, Settat and Marrakech. NASA POWER daily weather covers January 2015 to June 2024, yielding 10,407 observations. A continuous water-balance model with assumed soil parameters produces bounded proxy labels. Predictions use thirty consecutive days of weather and twenty-five features; field-measured crop stress is not available.

The additional DataOps pathway integrates dlt ingestion, DuckDB storage, dbt transformations, Python and SQL data contracts, and Dagster orchestration. It reproduces the original canonical weather and feature datasets. The MLOps workflow includes chronological model selection, MLflow tracking and registry, FastAPI serving, a Streamlit client, Docker Compose, GitHub Actions and monitoring reports. Histogram gradient boosting achieves a test RMSE of 0.188591 and R^2 of 0.625080, while a persistence comparator using full simulated state performs better. The report therefore emphasizes reproducibility and transparent scientific limitations rather than claiming field-validated irrigation support.

Keywords: crop water stress; wheat; weather data; DataOps; MLOps; data contract; model registry; chronological evaluation.

Liste des abréviations

API	Interface de programmation applicative
CI / CD	Intégration continue / livraison ou déploiement continus
EDA	Analyse exploratoire des données
ET0 / ETc	Évapotranspiration de référence / de la culture
FC / WP	Capacité au champ / point de flétrissement
Ks / Kc	Coefficient de stress / coefficient cultural
LST	Temps solaire local
MAE / RMSE	Erreur absolue moyenne / racine de l'erreur quadratique moyenne
PSI / KS	Population Stability Index / Kolmogorov–Smirnov
PR	Pull Request
TAW / RAW	Eau totale disponible / eau facilement utilisable
SHA-256	Fonction d'empreinte cryptographique

Table des matières

Table automatique. Dans Microsoft Word : Ctrl+A, puis F9 pour actualiser les champs après modification.

Introduction générale.....	1
Chapitre 1 — Contexte et problématique.....	2
1.1 Stress hydrique et objectif de prévision.....	2
1.2 Problématique et périmètre.....	2
1.3 Objectifs du projet.....	2
Chapitre 2 — Gestion Agile et collaboration.....	3
2.1 Méthode de reconstruction.....	3
2.2 Product Backlog et User Stories.....	3
2.3 Planification, Review et Retrospective.....	4
2.4 Git, GitHub et Pull Requests.....	4
Chapitre 3 — Données et DataOps.....	5
3.1 Source et couverture.....	5
3.2 Variables et unités.....	5
3.3 Ingestion avec dlt.....	6
3.4 Stockage DuckDB et transformations dbt.....	6
3.5 Orchestration Dagster.....	7
3.6 Chaîne DataOps et articulation MLOps.....	7
Chapitre 4 — Qualité, Data Contract et lineage.....	8
4.1 Stratégie de validation.....	8
4.2 Contrat canonique et garanties.....	8
4.3 Tests automatisés et données de test.....	9
4.4 Lineage et versionnement des données.....	9
Chapitre 5 — Exploration et feature engineering.....	10
5.1 Exploration réservée à l'apprentissage.....	10
5.2 Corrélations et limites d'interprétation.....	11
5.3 Construction des 25 variables explicatives.....	11
Chapitre 6 — Modélisation Machine Learning.....	12
6.1 Formulation et construction de la cible.....	12
6.2 Partition chronologique.....	12
6.3 Modèles testés et sélection.....	13
6.4 Évaluation finale et comparaison critique.....	14
Chapitre 7 — MLOps et MLflow.....	15
7.1 Suivi des expériences et artefacts.....	15
7.2 Registre, candidat et production.....	15

Chapitre 8 — API, interface et serving	16
8.1 FastAPI et contrat de requête	16
8.2 Erreurs, réponse et journalisation	16
8.3 Streamlit, client du service de prédiction	17
Chapitre 9 — Conteneurisation et déploiement	18
9.1 Services et réseau Docker.....	18
9.2 Komodo et limites de preuve.....	18
Chapitre 10 — Intégration continue	19
10.1 Workflows GitHub Actions.....	19
10.2 Résultats distants vérifiés	19
10.3 Ce que la CI ne réalise pas	19
Chapitre 11 — Monitoring et observabilité.....	20
11.1 Informations observées.....	20
11.2 Événements de service et absence de labels.....	20
11.3 Rapports historiques, simulés et événements réels.....	21
11.4 Observabilité opérationnelle.....	21
Chapitre 12 — Résultats, limites et discussion	22
12.1 Bilan scientifique et opérationnel	22
12.2 Priorités raisonnables	22
Conclusion générale	23
Bibliographie et sources	24
Annexes.....	25
Annexe A — Architecture et frontières opérationnelles	25
Annexe B — Arborescence fonctionnelle simplifiée	25
Annexe C — Contrat des endpoints FastAPI	26
Annexe D — Commandes de démonstration	26
Annexe E — Synthèse du Data Contract	27
Annexe F — Tests et modèles dbt.....	27
Annexe G — Assets et jobs Dagster	28
Annexe H — Docker Compose.....	28
Annexe I — Intégration continue et vérifications	29
Annexe J — Exemple JSON de référence.....	29
Annexe J — Requête JSON complète.....	30
Annexe J — Requête JSON complète (suite).....	31

Liste des figures

Figure 1 — Graphe des assets Dagster et séparation de l'entraînement explicite.....	7
Figure 2 — Architecture intégrée : chemin DataOps, apprentissage et services.....	7
Figure 3 — Traçabilité des données, du fichier source à l'événement de prédiction.....	9
Figure 4 — Distributions des variables et du proxy sur l'ensemble d'apprentissage.	10
Figure 5 — Évolution du proxy pendant les saisons d'apprentissage, par site.	10
Figure 6 — Matrice des corrélations de Pearson sur l'apprentissage.....	11
Figure 7 — Comparaison des RMSE de validation des quatre modèles.....	13
Figure 8 — Prédictions du modèle figé comparées au proxy sur le test.	14
Figure 9 — Capture existante du suivi d'expériences MLflow.....	15
Figure 10 — Documentation Swagger des endpoints réels.....	16
Figure 11 — Mode Automatic Weather avec instantané NASA vérifié en cache.	17
Figure 12 — Affichage de la prévision dans l'interface Streamlit.....	17
Figure 13 — Services Docker Compose et séparation du Dagster local.....	18

Liste des tableaux

Tableau 1 — Périmètre scientifique et fonctionnel	2
Tableau 2 — Product Backlog reconstitué à partir des fonctionnalités.....	3
Tableau 3 — Trois sprints pédagogiques et éléments de revue	4
Tableau 4 — Sites et hypothèses de sol	5
Tableau 5 — Variables météorologiques du schéma réel	5
Tableau 6 — Relations et responsabilités du stockage analytique	6
Tableau 7 — Règles numériques reprises du contrat existant.....	8
Tableau 8 — Couverture des contrôles automatisés	9
Tableau 9 — Familles de variables effectivement utilisées	11
Tableau 10 — Partitions réellement présentes après filtrage saisonnier	12
Tableau 11 — Hyperparamètres principaux de l’expérience	13
Tableau 12 — Comparaison réelle des modèles sur la validation.....	13
Tableau 13 — Résultats sur les 1 074 observations de test.....	14
Tableau 14 — Performance du modèle sélectionné par site	14
Tableau 15 — Métadonnées de la version de production	15
Tableau 16 — Endpoints et comportements de l’API.....	16
Tableau 17 — Services Compose, ports par défaut et persistance	18
Tableau 18 — Étapes réelles des workflows CI.....	19
Tableau 19 — Exécutions GitHub Actions du commit final audité.....	19
Tableau 20 — Indicateurs et seuils de monitoring configurés	20
Tableau 21 — Lecture des rapports de monitoring présents dans le dépôt	21
Tableau 22 — Limites, conséquences et pistes d’amélioration.....	22
Tableau 23 — Emplacements et responsabilités du dépôt	25
Tableau 24 — Champs de la réponse de prédiction	26
Tableau 25 — Commandes du Makefile et effets à connaître	26
Tableau 26 — Contrat structurel, temporel et de provenance.....	27
Tableau 27 — Dépendances et effets des assets.....	28
Tableau 28 — Matrice finale de conformité aux exigences du module	29

Introduction générale

La disponibilité de l'eau constitue une contrainte majeure pour les systèmes agricoles. Une gestion raisonnée suppose de comprendre l'équilibre entre les apports, la demande atmosphérique et la réserve du sol. La prédiction peut contribuer à cette compréhension, à condition que les informations utilisées, la définition de la cible et les limites du modèle soient explicites. Dans ce projet, l'objectif n'est pas de prescrire une dose d'irrigation, mais de construire un système reproductible de prévision d'un indicateur de stress hydrique modélisé pour le blé.

Le travail porte sur trois sites marocains : Meknès, Settat et Marrakech. Des observations météorologiques quotidiennes NASA POWER alimentent un bilan hydrique simplifié. Ce simulateur produit une cible comprise entre zéro et un ; le modèle apprend à prévoir sa valeur du lendemain à partir de trente jours d'historique. Les résultats mesurent donc l'accord avec un proxy, et non une précision agronomique validée au champ.

La problématique dépasse l'ajustement d'un estimateur. Une prévision exploitable doit être associée à des données identifiables, des contrôles de qualité, une évaluation temporelle, une version de modèle et une chaîne de service vérifiable. Le projet articule ainsi DataOps et MLOps : acquisition, transformations, contrat de données, orchestration, entraînement, suivi d'expériences, API, interface et monitoring.

La démarche retenue consiste à conserver un pipeline Python fonctionnel et à lui ajouter un chemin DataOps parallèle. Les deux chemins reproduisent le même format canonique et les mêmes variables explicatives. Cette séparation permet de démontrer dlt, DuckDB, dbt et Dagster sans rendre ces outils obligatoires au démarrage de l'application.

Le présent rapport repose sur l'audit du dépôt au commit 0f3bc50, le 17 septembre 2026. Les métriques proviennent des artefacts versionnés. Les captures sont celles du dépôt ; les nouveaux schémas sont des représentations de l'implémentation, pas des captures d'interfaces. Les éléments Agile non documentés sont identifiés comme une reconstruction pédagogique. Les chapitres suivent le cycle de vie des données jusqu'à la discussion des résultats.

Source : README.md ; configs/config.yaml ; docs/methodology.md ; docs/dataops-evidence.json.

Chapitre 1 — Contexte et problématique

1.1 Stress hydrique et objectif de prévision

Le stress hydrique traduit une limitation de la satisfaction des besoins en eau d'une culture. Son observation peut mobiliser des mesures du sol ou de la plante ; aucune mesure de ce type n'est fournie ici. Le projet représente cette limitation au moyen d'un réservoir hydrique simulé et d'un coefficient de réduction de l'évapotranspiration. La référence FAO-56 fournit le cadre méthodologique, tandis que les paramètres de sol sont des hypothèses de scénario [R2].

1.2 Problématique et périmètre

La question étudiée est la suivante : comment prévoir l'indice de stress modélisé du lendemain à partir de la météo disponible, tout en assurant la traçabilité, la qualité et la reproductibilité de la chaîne ? Le périmètre est volontairement limité afin de rendre l'ensemble vérifiable dans un contexte universitaire.

Tableau 1 — Périmètre scientifique et fonctionnel

Dimension	Choix réellement implémenté	Conséquence
Culture	Blé pluvial, identifiant wheat	Pas de modèle multi-cultures
Sites	meknes, settat, marrakech	Pas de généralisation géographique démontrée
Météo	01/01/2015–30/06/2024	Séries historiques quotidiennes
Horizon	Un jour, à partir de la fin du jour courant	Météo du lendemain exclue des entrées
Historique API	Exactement 30 jours consécutifs	Ordre et couverture contrôlés
Cible	next_day_1_minus_Ks_proxy	Indice simulé, sans unité, entre 0 et 1
Saison	Semis le 15 novembre, durée 180 jours	Origine et cible doivent être en saison
État initial	365 jours de stabilisation	Première année exclue de l'apprentissage

1.3 Objectifs du projet

L'objectif scientifique est de comparer quatre modèles sur une partition chronologique commune. L'objectif technique est d'assurer un chemin auditable depuis la source jusqu'à la prédiction. L'objectif pédagogique est de démontrer la complémentarité des pratiques DataOps, MLOps, de conteneurisation et d'intégration continue.

Les données ne permettent pas de conclure à une économie d'eau, à un gain de rendement ou à une efficacité réelle d'irrigation. Ces résultats nécessiteraient un protocole de terrain. Le système constitue une base expérimentale et logicielle, avec un domaine d'application explicitement borné.

Source : configs/config.yaml ; src/features/water_balance.py ; src/api/schemas.py ; docs/methodology.md.

Chapitre 2 — Gestion Agile et collaboration

2.1 Méthode de reconstruction

L'historique Git atteste des livraisons techniques, mais aucun journal de cérémonies Scrum, calendrier de sprint ou relevé de participation des sept étudiants n'a été trouvé. Les trois sprints ci-dessous constituent une lecture rétrospective de ces livraisons. Ils ne sont pas présentés comme la preuve de réunions réellement tenues. Les responsabilités individuelles restent à renseigner par l'équipe.

2.2 Product Backlog et User Stories

Le backlog formalise les besoins couverts par le produit. La priorité et l'affectation aux sprints sont proposées pour structurer la présentation académique ; le statut est fondé sur les fichiers présents. Chaque récit relie un utilisateur, une fonctionnalité et une finalité vérifiable.

Tableau 2 — Product Backlog reconstitué à partir des fonctionnalités

ID	User Story	Priorité	Sprint	Statut technique
US0 1	En tant qu'analyste, je souhaite des séries NASA traçables afin de reproduire les données.	Haute	S1	Implémenté
US0 2	En tant que data scientist, je souhaite valider les unités et les dates afin d'éviter des entrées incohérentes.	Haute	S1	Implémenté
US0 3	En tant que data scientist, je souhaite comparer les modèles chronologiquement afin de limiter les fuites temporelles.	Haute	S1	Implémenté
US0 4	En tant que responsable ML, je souhaite versionner candidats et production afin de contrôler les mises en service.	Haute	S1	Implémenté
US0 5	En tant qu'utilisateur, je souhaite soumettre 30 jours de météo via une interface afin d'obtenir une prévision.	Haute	S2	Implémenté
US0 6	En tant qu'utilisateur, je souhaite un mode automatique avec cache afin de limiter la saisie et les pannes de démonstration.	Haute	S2	Implémenté
US0 7	En tant qu'exploitant, je souhaite démarrer les services par Compose afin de reproduire le déploiement.	Haute	S2	Implémenté localement
US0 8	En tant qu'ingénieur données, je souhaite dlt, DuckDB et dbt afin de tracer les transformations.	Haute	S3	Implémenté
US0 9	En tant qu'opérateur, je souhaite visualiser les dépendances Dagster afin de suivre les exécutions.	Haute	S3	Implémenté
US1 0	En tant que mainteneur, je souhaite des tests CI hors ligne afin de contrôler les régressions.	Haute	S3	Deux workflows présents
US1 1	En tant que membre de l'équipe, je souhaite une revue par PR afin de discuter les changements avant fusion.	Moyenne	S3	Procédure ; aucune PR observée

La définition de « terminé » retenue pour cette reconstruction associe une implémentation, un artefact ou un test et une documentation. Elle distingue donc un code disponible d'une validation distante ou agronomique, qui demande une preuve supplémentaire.

Source : `git log --all ; tests/ ; README.md ; docs/dataops-pull-request.md`.

2.3 Planification, Review et Retrospective

Le sprint S1 regroupe le socle scientifique et MLOps : collecte, proxy, variables, comparaison et suivi MLflow. Le sprint S2 correspond à l'interface, au mode météo automatique et à la consolidation du déploiement. Le sprint S3 ajoute la voie DataOps et sa CI. Les dates citées ci-dessous sont des dates de commits ; elles ne définissent pas la durée réelle du travail.

Tableau 3 — Trois sprints pédagogiques et éléments de revue

Sprint	Preuves Git	Sprint Review : livrable observable	Retrospective proposée
S1 — Socle	55bc486, 62a3218, 2781f80 ; 07/09/2026	Pipeline, comparaison, artefacts, documentation	Clarifier le proxy et conserver un test temporel figé
S2 — Application	13f7318, bb8b80f ; 07/09 ; b9b8d88, 5fc1fa9 ; 08/09 ; 85f9ace ; 09/09	Streamlit, cache météo, Compose et captures	Préférer des instantanés vérifiés et documenter les ports hôtes
S3 — DataOps	5583615, 0f3bc50 ; 17/09/2026	Ingestion dlt, SQL dbt, contrat, Dagster et CI	Isoler les dépendances et vérifier l'équivalence avec le chemin initial

Les Reviews proposées reposent sur une démonstration de livrables : requête d'exemple, tests, tables DuckDB et graphe Dagster. Les Retrospectives sont des analyses rédigées à partir des choix techniques et des corrections visibles. Aucun verbatim de réunion, estimation de vélocité ou engagement individuel n'est inventé.

2.4 Git, GitHub et Pull Requests

Le dépôt distant est Anass-Erf/MLOPS_Project. La branche audité feature/dataops-pipeline est suivie par origin/feature/dataops-pipeline au commit 0f3bc50 ; main est référencée à 85f9ace dans les références locales. L'historique établit une progression du produit, mais ne prouve pas la répartition du travail entre les sept étudiants.

L'API GitHub consultée le 17 septembre 2026 renvoie une liste vide pour les Pull Requests, tous états confondus. La procédure de création d'une PR est documentée, mais aucune revue effective ni fusion par PR ne peut être revendiquée. En revanche, les deux workflows CI du commit 0f3bc50 sont publiquement terminés avec le statut success [R14, R15].

Certaines notes conservées dans docs/dataops-evidence.json décrivent un état antérieur à cette publication. Pour la situation Git et CI, le rapport donne priorité à l'historique actuel et aux résultats GitHub datés. Cette distinction évite de confondre un compte rendu historique avec l'état final du dépôt.

Source : git log ; git branch -avv ; rapports/sources/verification_github.json ; docs/dataops-pull-request.md.

Chapitre 3 — Données et DataOps

3.1 Source et couverture

NASA POWER est la seule source de données météorologiques effectivement utilisée. Le collecteur interroge l'API quotidienne ponctuelle avec la communauté AG et le temps solaire local LST [R1]. Les sept paramètres demandés sont conservés avec les métadonnées de réponse. Les trois instantanés versionnés couvrent 3 469 jours par site, soit 10 407 observations.

La FAO intervient comme référence méthodologique du bilan hydrique, pas comme source d'un jeu de données de sols téléchargé. WaPOR et Google Earth Engine sont évoqués dans la documentation comme pistes futures ; aucun adaptateur opérationnel n'alimente l'entraînement depuis ces plateformes.

Tableau 4 — Sites et hypothèses de sol

Site	Latitude	Longitude	Capacité au champ	Point de flétrissement
Meknès	33,89	-5,55	0,30 m ³ /m ³	0,15 m ³ /m ³
Settat	33,00	-7,62	0,28 m ³ /m ³	0,14 m ³ /m ³
Marrakech	31,63	-7,99	0,25 m ³ /m ³	0,12 m ³ /m ³

3.2 Variables et unités

Tableau 5 — Variables météorologiques du schéma réel

Paramètre NASA	Colonne canonique	Unité du projet
T2M	temperature	°C
T2M_MAX	temperature_max	°C
T2M_MIN	temperature_min	°C
PRECTOTCORR	precipitation	mm/jour
RH2M	humidity	%
WS2M	wind_speed	m/s, à 2 m
ALLSKY_SFC_SW_DWN	solar_radiation	MJ/m ² /jour
Index quotidien	date	Date calendaire
Site configuré	site_id	meknes, settat ou marrakech

Les coordonnées, field_capacity et wilting_point complètent les colonnes météo pour constituer le jeu canonique à treize colonnes. Elles proviennent de la configuration. La profondeur racinaire est fixée à un mètre au stade des variables explicatives. Une radiation fournie en kWh/m²/jour est multipliée par 3,6 ; toute unité inconnue est rejetée. Les valeurs de remplissage NASA deviennent des valeurs manquantes, puis déclenchent les contrôles.

Source : src/data/collect_data.py ; src/data/preprocess.py ; configs/config.yaml ; data/raw/*.manifest.json.

3.3 Ingestion avec dlt

Le module `dlt_pipeline.py` réutilise `collect`, `verify_snapshot` et `normalize_payload`. Il ne crée pas un fournisseur météo concurrent. Avant chargement, il vérifie l’empreinte du fichier, l’origine NASA, l’URL et les paramètres de requête, puis la couverture et les règles météo. L’exécution est hors ligne par défaut ; l’option `--live` autorise le téléchargement d’un instantané configuré manquant.

Le pipeline `dlt nasa_weather` charge `raw.raw_weather` dans DuckDB. Le mode `replace` recharge les observations validées au lieu d’accumuler des doublons. Ce choix convient au petit corpus historique complet ; il ne constitue pas une ingestion incrémentale en continu. Les fichiers JSON sources restent immuables. Les valeurs chargées sont déjà dans les unités canoniques, même si les colonnes brutes conservent des noms NASA en minuscules.

3.4 Stockage DuckDB et transformations dbt

Tableau 6 — Relations et responsabilités du stockage analytique

Relation	Rôle	Informations caractéristiques
<code>raw.raw_weather</code>	Destination dlt	<code>observed_on</code> , <code>site_id</code> , sept variables NASA normalisées, scénario, provenance
<code>weather_staging.stg_weather</code>	Table dbt de staging	date typée, noms canoniques, valeurs double, provenance
<code>weather_analytics.ml_weather</code>	Table dbt destinée au ML	Projection des treize colonnes météo/scénario
<code>raw._dlt_loads</code>	Métadonnées dlt	Identification des chargements
<code>raw._dlt_pipeline_state</code> et <code>raw._dlt_version</code>	État technique dlt	État du pipeline et schéma

La base générée est `artifacts/dataops/crop_water_stress.duckdb`. Aucun serveur de base externe ni secret n’est nécessaire. `dbt-duckdb` exécute réellement les transformations. Le modèle `stg_weather` effectue des conversions de types strictes et le renommage ; `ml_weather` produit la projection attendue par le pipeline Python. Les deux modèles sont matérialisés en tables et déclarent un contrat de schéma `enforced`.

Le nettoyage n’élimine pas silencieusement les anomalies : ni interpolation, ni suppression de lignes invalides, ni déduplication SQL ne masque les défauts. Une erreur entraîne un échec contrôlé. Les fenêtres temporelles et le calcul de la cible restent dans Python, où l’implémentation est partagée avec le service de prédiction.

DuckDB est adapté à cette analyse locale, mais les opérations d’écriture ne doivent pas être exécutées simultanément depuis plusieurs processus indépendants. Le Makefile propose des commandes explicites d’ingestion, de construction dbt, de qualité et d’inspection.

Source : `src/dataops/dlt_pipeline.py` ; `dbt_project/` ; `src/dataops/dbt_runner.py` ; Makefile. Références : [R5–R7].

3.5 Orchestration Dagster

Dagster représente les étapes par des assets et enregistre leurs dépendances, journaux et matérialisations. La source externe `nasa_snapshots` précède six assets exécutables : `ingest_weather`, `dbt_transform`, `data_quality`, `build_features`, `train_model` et `evaluate_model`. Le graphe ci-dessous reprend ces noms réels.

Cette étape est illustrée à la figure 1.

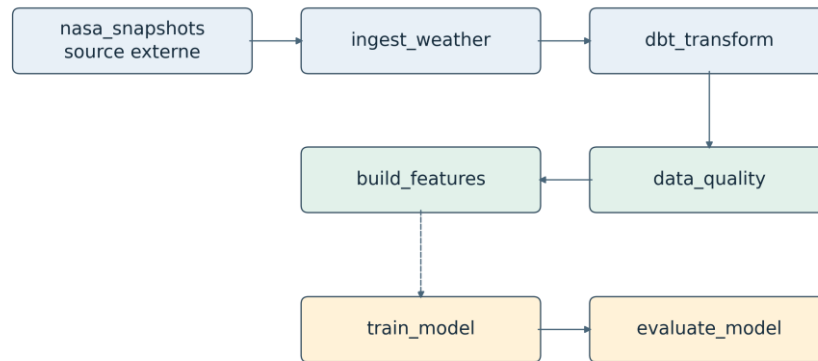


Figure 1 — Graphe des assets Dagster et séparation de l'entraînement explicite.

Source : Schéma établi à partir de `src/dataops/definitions.py`.

Le job `dataops_demo` sélectionne les quatre premières étapes exécutables et s'arrête après la génération des variables. Le job `train_evaluate_explicit` sélectionne entraînement et évaluation ; il suppose les features déjà produites. La configuration de l'asset d'entraînement est désactivée par défaut hors de ce consentement explicite. Aucun asset de promotion et aucun calendrier automatique ne sont définis.

3.6 Chaîne DataOps et articulation MLOps

Cette étape est illustrée à la figure 2.

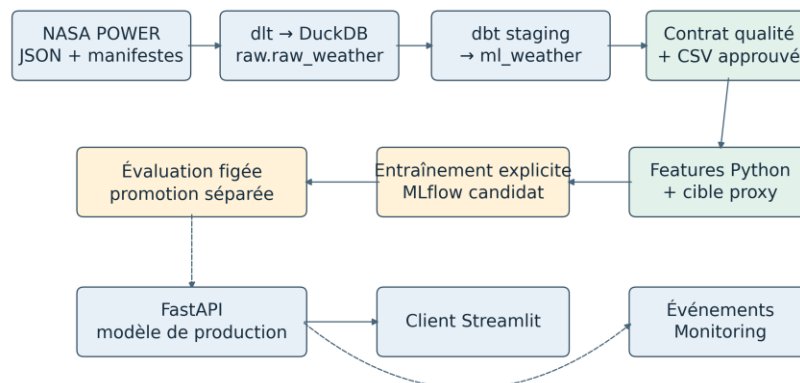


Figure 2 — Architecture intégrée : chemin DataOps, apprentissage et services.

Source : Schéma de synthèse du code ; la liaison vers la production exige une opération de mise en service distincte.

Dagster pilote le traitement ; il n'est pas une transformation supplémentaire placée après la qualité. Les preuves conservées attestent deux exécutions réussies. Le chemin de démonstration prend environ 19 secondes dans la vérification documentée, sans garantie de durée sur une autre machine. L'environnement `.venv-dataops` est séparé de l'environnement ML ; les fonctions de modélisation sont appelées avec l'interpréteur initial.

Source : `src/dataops/definitions.py` ; `src/dataops/cli.py` ; `src/dataops/settings.py` ; `docs/dataops-evidence.json`. Référence : [R8].

Chapitre 4 — Qualité, Data Contract et lineage

4.1 Stratégie de validation

La qualité est contrôlée à plusieurs frontières : instantané source, normalisation Python, tables dbt, export canonique, dataset de features et requête API. Les contrôles de structure évitent les incohérences techniques ; les bornes et relations entre variables empêchent des valeurs incompatibles avec le contrat choisi. Ces bornes sont celles du code et ne constituent pas des recommandations agronomiques.

Tableau 7 — Règles numériques reprises du contrat existant

Variable	Intervalle autorisé	Justification dans le système
temperature	[-60 ; 60] °C	Domaine de validité des entrées
temperature_max	[-60 ; 65] °C	Contrôle des extrêmes
temperature_min	[-70 ; 60] °C	Contrôle des extrêmes
precipitation	[0 ; 1 000] mm/jour	Absence de pluie négative
humidity	[0 ; 100] %	Humidité relative bornée
wind_speed	[0 ; 75] m/s	Vitesse non négative et bornée
solar_radiation	[0 ; 50] MJ/m ² /jour	Radiation normalisée et bornée
Températures	minimum ≤ moyenne ≤ maximum	Cohérence d'une même journée
Sol	0 < wilting_point < field_capacity < 1	Réserve utile strictement positive
root_depth_m	[0,1 ; 3] m	Paramètre de sol admis par validate_soil

4.2 Contrat canonique et garanties

Le contrat DataOps exige exactement les treize colonnes dans leur ordre canonique. Il réutilise `validate_weather`, vérifie tous les sites attendus et la couverture complète, puis exige des coordonnées et paramètres de sol identiques à la configuration. Le contrôle final compare chaque valeur météo à la normalisation indépendante des instantanés vérifiés. La provenance de staging doit également correspondre aux fichiers sources.

4.2.1 Publication et consommation du jeu approuvé

L'écriture du CSV est atomique : un fichier temporaire remplace la sortie seulement après validation. Un rapport `quality.json` contient l'empreinte approuvée. Le worker ML refuse un CSV modifié après approbation et refuse des features fondées sur une ancienne empreinte météo. Ces barrières relient la qualité à la consommation effective des données, au-delà d'un simple affichage de tests verts.

Source : `src/data/validate_data.py` ; `src/dataops/contract.py` ; `src/dataops/ml_bridge.py` ; `src/api/schemas.py`.

4.3 Tests automatisés et données de test

Les tests combinent règles SQL, assertions Python et scénarios d'échec. Les fixtures artificielles sont réservées aux tests logiciels ; les résultats scientifiques utilisent les observations NASA et le proxy défini. Le fichier de preuve versionné rapporte 70 tests initiaux et 21 tests DataOps réussis. Dans l'environnement applicatif sans dépendances optionnelles, le module DataOps est ignoré ; il est exécuté séparément dans son environnement dédié.

Tableau 8 — Couverture des contrôles automatisés

Couche	Contrôles présents	Preuve
Sources	Empreinte, requête, manifeste absent	tests/dataops/test_dataops.py
Météo Python	Colonnes, types, valeurs finies, dates, doublons, bornes	tests/test_preprocessing.py
dbt	30 not_null, 2 accepted_values, 3 tests SQL métier	dbt_project/models/schema.yml et tests/
Features	Causalité, isolation des sites, parité historique/API	tests/test_features.py
Modèle/API	Bornage, altération d'artefact, succès et erreurs HTTP	tests/test_model.py ; tests/test_api.py
Interface	Exemple, deux modes, cache, indisponibilité NASA	tests/test_ui.py ; tests/test_weather_ui.py
Orchestration	Définition des jobs, entraînement volontaire, features obsolètes	tests/dataops/test_dataops.py

4.4 Lineage et versionnement des données

Cette étape est illustrée à la figure 3.

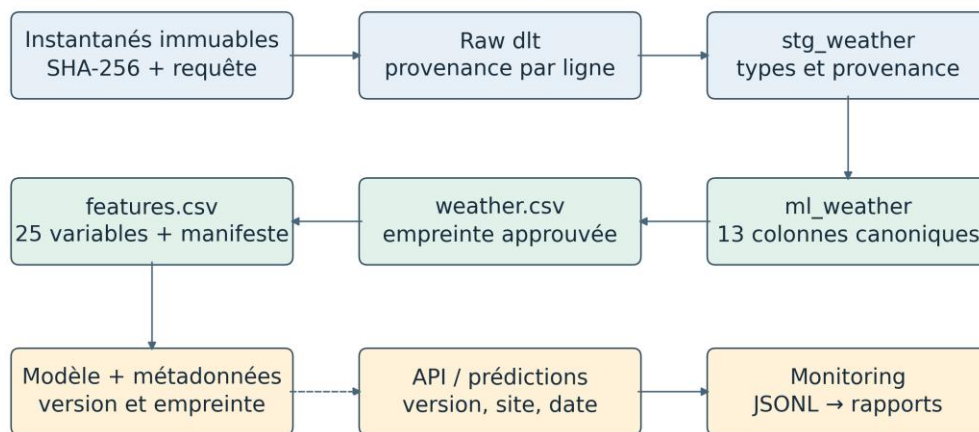


Figure 3 — Traçabilité des données, du fichier source à l'événement de prédiction.

Source : Schéma construit à partir des manifestes, des contrats et des fonctions de provenance.

Le lineage SQL est exprimé par source et ref dans dbt ; Dagster expose les dépendances entre étapes. Les reçus de provenance relient fichiers d'entrée et de sortie, configuration, code et version Python. Les empreintes SHA-256 identifient les contenus ; elles ne remplacent ni un système d'archivage complet, ni une sauvegarde, ni une validation scientifique. Le dépôt n'implémente pas DVC et ne prétend pas fournir son stockage distant ou son cache de dépendances.

Source : src/utis/provenance.py ; artifacts/provenance/ ; artifacts/dataset_manifest.json ; docs/dataops-evidence.json.

Chapitre 5 — Exploration et feature engineering

5.1 Exploration réservée à l'apprentissage

Le notebook 01_data_exploration.ipynb et src/reports.py examinent les 3 081 lignes d'apprentissage. Les distributions et corrélations n'utilisent pas les sorties de validation ou de test pour guider les choix. Le rapport de qualité indique zéro valeur manquante après validation et zéro doublon sur les 10 407 observations normalisées. Ce résultat provient des contrôles, et non d'une imputation.

Cette étape est illustrée à la figure 4.

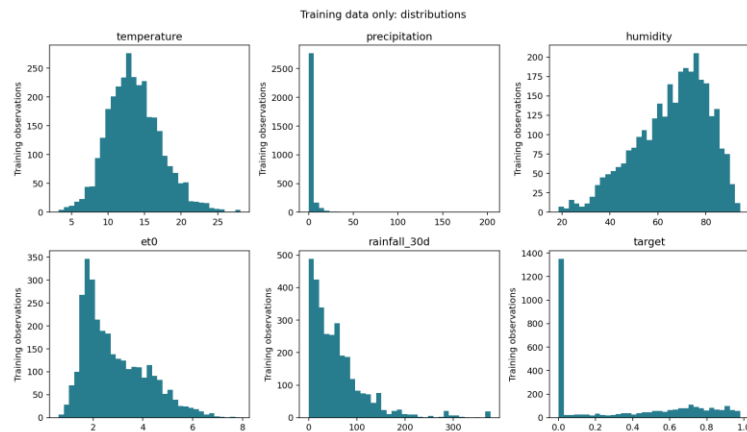


Figure 4 — Distributions des variables et du proxy sur l'ensemble d'apprentissage.

Source : Artefact existant : artifacts/eda/distributions.png.

La moyenne de la cible d'apprentissage vaut 0,34717 ; 43,30 % des valeurs sont exactement nulles. Cette masse en zéro rend la MAPE peu appropriée et rappelle le caractère borné du proxy. La pluie est asymétrique : sa médiane d'apprentissage vaut 0,04 mm/jour, alors que sa moyenne vaut environ 1,965 mm/jour.

Cette étape est illustrée à la figure 5.

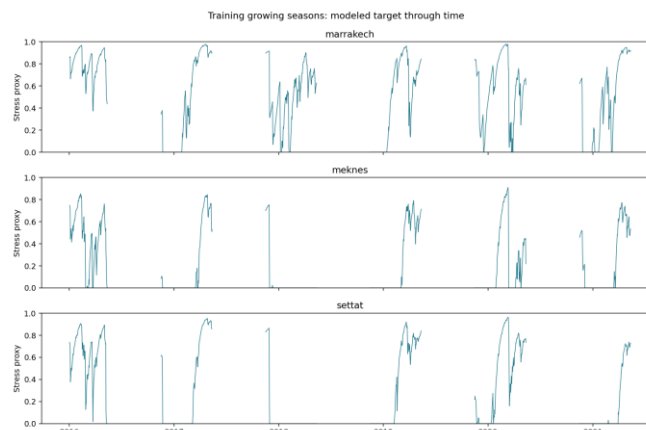


Figure 5 — Évolution du proxy pendant les saisons d'apprentissage, par site.

Source : Artefact existant ; les interruptions entre saisons sont laissées visibles.

Les discontinuités graphiques hors saison ne signalent pas des observations météo manquantes. Elles résultent de la sélection des journées où l'origine et la cible appartiennent à la saison active. Les extrêmes physiquement admis sont conservés : les bornes IQR sont descriptives et ne déclenchent pas une suppression automatique.

Source : notebooks/01_data_exploration.ipynb ; artifacts/eda/summary.json ; artifacts/eda/descriptive_statistics.csv ; src/reports.py.

5.2 Corrélations et limites d'interprétation

Cette étape est illustrée à la figure 6.

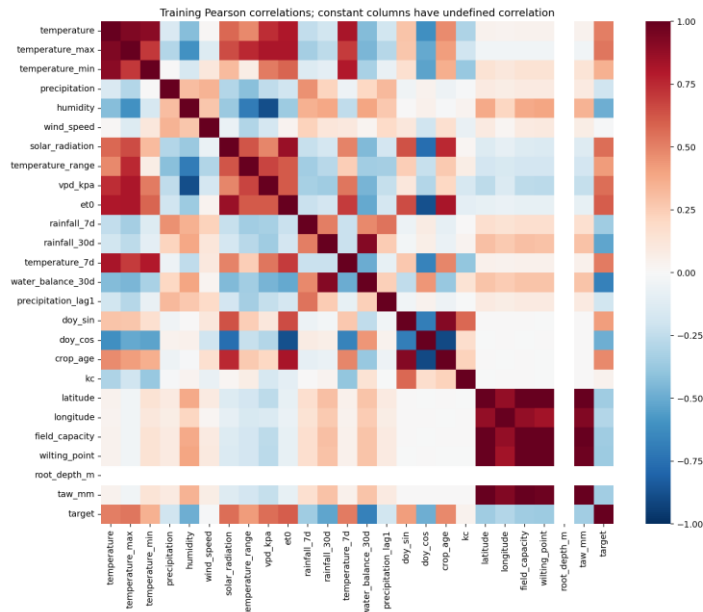


Figure 6 — Matrice des corrélations de Pearson sur l'apprentissage.

Source : Artefact existant ; une variable constante possède une corrélation indéfinie.

La corrélation agrège ici des journées et des sites différents. Elle ne démontre pas un lien causal : les coordonnées et les hypothèses de sol sont constantes par site et peuvent refléter des différences de climat. La profondeur racinaire est constante dans tout le scénario. Les flags IQR comptabilisent notamment 552 précipitations et 149 cumuls de pluie à trente jours atypiques, sans en déduire qu'ils sont erronés.

5.3 Construction des 25 variables explicatives

Tableau 9 — Familles de variables effectivement utilisées

Famille	Variables	Disponibilité
Météo du jour	Sept variables météorologiques	Fin du jour t
Atmosphère	temperature_range, vpd_kpa, et0	Calcul déterministe à partir de t
Historique	rainfall_7d, rainfall_30d, temperature_7d, water_balance_30d, precipitation_lag1	Fenêtres causales par site
Calendrier	doy_sin, doy_cos, crop_age, kc	Saison connue à l'avance
Scénario	latitude, longitude, field_capacity, wilting_point, root_depth_m, taw_mm	Configuration statique

Les fenêtres incluent le jour courant et excluent le futur. Les vingt-neuf premières lignes d'un historique ne possèdent pas une fenêtre complète. La même fonction `weather_features` construit les entrées d'entraînement et la dernière ligne utilisée en API. `stress_today`, l'humidité simulée du sol et le déficit hydrique sont exclus des features afin de ne pas fournir directement l'état servant à calculer la cible.

Source : `src/features/build_features.py` ; `tests/test_features.py` ; `artifacts/eda/iqr_outliers.csv`.

Chapitre 6 — Modélisation Machine Learning

6.1 Formulation et construction de la cible

Le problème est une régression supervisée bornée. À l'origine t , le modèle utilise la météo connue jusqu'à t et prédit $y(t+1) = 1 - Ks(t+1)$. Le coefficient Ks dépend du déficit en eau simulé dans le réservoir racinaire. Le bilan est calculé continûment, y compris entre les saisons ; il n'est pas réinitialisé chaque année à un sol humide.

La réserve totale disponible est $TAW = 1\,000 \times (FC - WP) \times Z_r$, en millimètres. Le scénario retient 80 % des précipitations comme apport effectif, un déficit initial de 50 % de TAW et une profondeur de un mètre. Le coefficient cultural varie selon quatre phases de 20, 40, 70 et 50 jours. L'évapotranspiration de référence est approximée par Hargreaves–Samani, avec conversion du rayonnement extraterrestre par le facteur 0,408.

Chaque journée applique la pluie, le drainage de l'excédent, puis une évapotranspiration limitée par l'eau disponible. Les états simulés sont sauvegardés dans `proxy_audit.csv`. La météo du lendemain sert à construire rétrospectivement le label de demain ; elle n'entre jamais dans les variables de prévision du jour courant. Les tests de perturbation du futur vérifient cette séparation.

6.2 Partition chronologique

Tableau 10 — Partitions réellement présentes après filtrage saisonnier

Ensemble	Règle sur <code>target_date</code>	Première cible retenue	Dernière cible retenue	Lignes
Apprentissage	Jusqu'au 30/06/2021	02/01/2016	13/05/2021	3 081
Validation	01/07/2021–30/06/2022	16/11/2021	13/05/2022	537
Test	Après le 30/06/2022	16/11/2022	12/05/2024	1 074

Les bornes sont communes à tous les sites. Un découpage aléatoire mélangerait des journées voisines et des états hydriques autocorrélés, ce qui rendrait l'évaluation temporelle moins représentative. L'historique antérieur à une frontière reste admissible puisqu'il aurait été disponible au moment de la prévision. Le protocole estime un transfert temporel sur des sites connus ; il ne démontre pas un transfert vers un nouveau site.

Source : `src/features/water_balance.py` ; `src/features/build_features.py` ; `artifacts/dataset_manifest.json` ; `docs/methodology.md`.

6.3 Modèles testés et sélection

Quatre configurations prédéfinies sont comparées : DummyRegressor à la moyenne, Ridge précédée d'un StandardScaler, RandomForestRegressor et HistGradientBoostingRegressor. Tous sont enveloppés dans BoundedRegressor, qui borne les sorties à [0 ; 1] pendant l'évaluation comme pendant le service. Le scaler est ajusté à l'intérieur du pipeline de Ridge, sur l'apprentissage uniquement.

Tableau 11 — Hyperparamètres principaux de l'expérience

Modèle	Configuration effectivement utilisée
dummy_mean	strategy=mean
ridge	StandardScaler ; alpha=10,0
random_forest	160 arbres ; min_samples_leaf=4 ; random_state=42 ; n_jobs=1
gradient_boosting	max_iter=200 ; max_leaf_nodes=15 ; learning_rate=0,05 ; early_stopping=False ; random_state=42

L'arrêt anticipé interne du boosting est désactivé pour éviter une partition aléatoire implicite. La RMSE de validation détermine le meilleur modèle ; la MAE puis le nom servent à départager une égalité. La configuration retenue est ensuite réajustée sur apprentissage et validation réunis. Le test reste réservé à l'évaluation finale du modèle figé.

Tableau 12 — Comparaison réelle des modèles sur la validation

Modèle	RMSE	MAE	R ²	Décision
HistGradientBoosting	0,170564	0,128789	0,617846	Sélectionné
Random Forest	0,177395	0,134563	0,586623	Non retenu
Ridge	0,203292	0,164718	0,457120	Non retenu
Dummy moyenne	0,326490	0,293414	-0,400239	Référence naïve

Cette étape est illustrée à la figure 7.

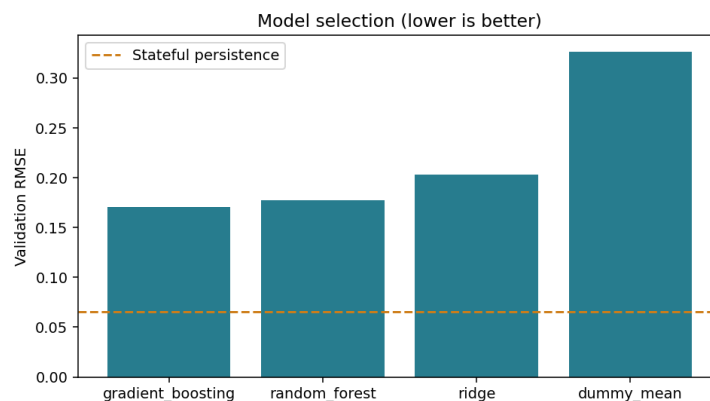


Figure 7 — Comparaison des RMSE de validation des quatre modèles.

Source : Artefact existant ; la ligne de persistance est un comparateur disposant de l'état simulé courant.

6.3.1 Interprétation des métriques

La MAE exprime l'erreur absolue moyenne sur l'indice ; la RMSE accorde davantage de poids aux grandes erreurs ; R² compare la variance résiduelle à celle d'une prédiction constante. Un R² négatif est possible, comme pour le modèle naïf. Aucun pourcentage de « précision » de classification n'est substitué à ces métriques de régression.

Source : src/models/estimators.py ; src/models/train.py ; artifacts/model_comparison.csv. Référence : [R9].

6.4 Évaluation finale et comparaison critique

Tableau 13 — Résultats sur les 1 074 observations de test

Approche	RMSE	MAE	R ²
Modèle sélectionné	0,188591	0,140200	0,625080
Persistance de l'état simulé	0,059734	0,027396	0,962387

La persistance prédit le stress du lendemain à partir de l'état simulé du jour courant. Elle bénéficie du bilan hydrique historique complet, alors que le ML dispose d'une fenêtre météo limitée et de paramètres statiques. Cette asymétrie ne doit pas masquer le résultat : la persistance est nettement meilleure dans cette expérience. Le projet ne démontre donc pas une supériorité prédictive du ML sur ce comparateur.

Cette étape est illustrée à la figure 8.

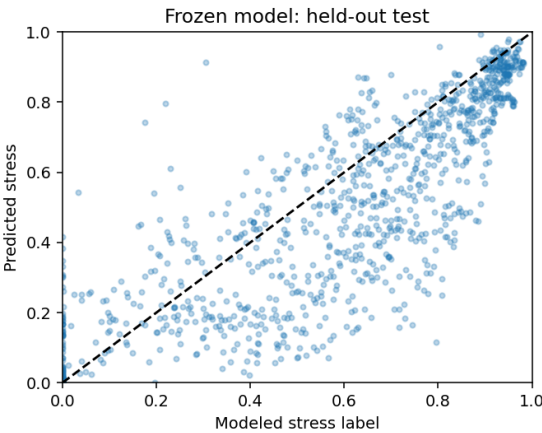


Figure 8 — Prédictions du modèle figé comparées au proxy sur le test.

Source : Artefact existant ; l'axe de référence représente des labels simulés, pas des mesures au champ.

Tableau 14 — Performance du modèle sélectionné par site

Site	RMSE	MAE	R ²
Marrakech	0,187582	0,132405	0,145725
Meknès	0,196833	0,151024	0,623866
Settat	0,181024	0,137170	0,589041

Le R² plus faible à Marrakech montre que la métrique globale ne résume pas uniformément les sites. Il n'autorise pas à attribuer la différence à un mécanisme causal sans analyse supplémentaire. Les valeurs rapportées sont des estimations ponctuelles sur des séries autocorrélées ; aucun intervalle de confiance par saison ou validation géographique n'est fourni.

L'évaluation réutilise un rapport figé lorsqu'il existe pour la version et l'empreinte du modèle. Une nouvelle optimisation après consultation du test nécessiterait une nouvelle période de réserve, afin de ne pas transformer progressivement le test en ensemble de sélection.

Source : src/models/evaluate.py ; artifacts/evaluation/v4-2771746c21f7/test_metrics.json.

Chapitre 7 — MLOps et MLflow

7.1 Suivi des expériences et artefacts

Le module train.py ouvre un run MLflow pour chaque configuration évaluée, puis un run candidat pour le modèle réajusté. Il journalise la graine, le type d'estimateur, les paramètres, l'empreinte du dataset, les variables utilisées, la configuration, les métriques et les graphiques. Le modèle est enregistré avec une signature et un exemple d'entrée. Le notebook 02_model_experiments.ipynb consulte ces résultats sans réentraîner automatiquement.

Cette étape est illustrée à la figure 9.

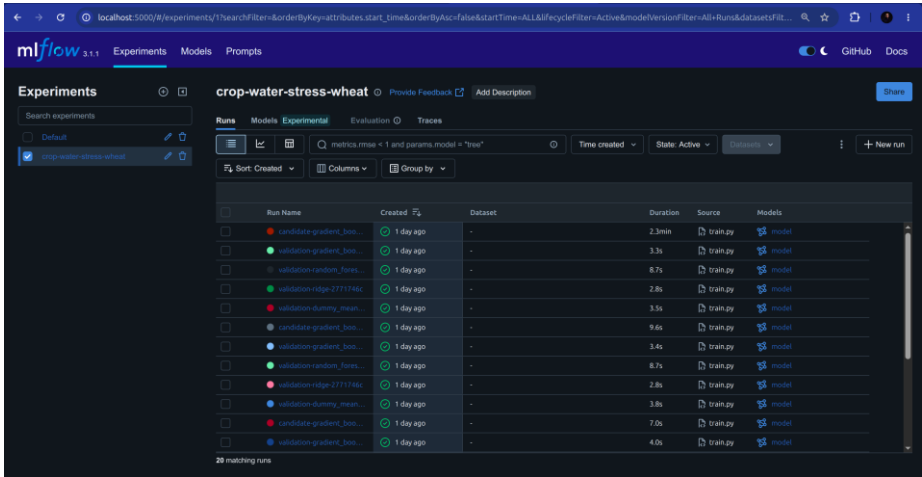


Figure 9 — Capture existante du suivi d'expériences MLflow.

Source : Capture du dépôt datée par son nom du 8 septembre 2026 ; elle illustre l'interface à cette date.

7.2 Registre, candidat et production

L'expérience initiale est crop-water-stress-wheat et le registre crop-water-stress. L'entraînement met à jour l'alias candidate ; la promotion est une commande distincte qui exige une évaluation cohérente avec l'empreinte du modèle. Les fichiers locaux et models/production.json permettent au service de fonctionner sans joindre MLflow à chaque prédiction.

Tableau 15 — Métadonnées de la version de production

Champ	Valeur vérifiée
Version locale	v4-2771746c21f7
Version du registre dans le pointeur	4
Modèle	gradient_boosting
Fichier	models/v4-2771746c21f7/model.joblib
Empreinte SHA-256	e8a7e2a6dfad19b4763d042694be00c4a72c234585579f4b3f8555110283fca4
Nombre de variables	25
Cycle de vie	Entraînement → candidat → évaluation figée → promotion explicite

La voie DataOps utilise un stockage et un registre séparés, crop-water-stress-dataops-demo. Elle ne remplace pas le modèle servi. Attention à la distinction entre commandes : make train seul ne promeut pas, tandis que make pipeline enchaîne explicitement l'entraînement, l'évaluation puis make promote. Le démarrage de l'API, de Streamlit, de MLflow ou de Compose ne déclenche aucun de ces entraînements.

Source : src/models/train.py ; src/models/registry.py ; src/models/common.py ; models/production.json ; Makefile. Référence : [R3].

Chapitre 8 — API, interface et serving

8.1 FastAPI et contrat de requête

FastAPI charge le modèle depuis un pointeur local et contrôle l’empreinte de son fichier. Le service reconstruit les mêmes variables explicatives que le traitement batch. L’utilisateur soumet des observations météo, pas une ligne de features précalculées, ce qui réduit les divergences entre entraînement et inférence.

Tableau 16 — Endpoints et comportements de l’API

Route	Fonction	Résultat attendu
GET /	Informations sur le projet et le contrat	Nom, cible, historique de 30 jours, blé
GET /health	Disponibilité du modèle	200 avec version ; 503 si indisponible
POST /predict	Validation et prédiction	Score, version, date, unité, niveau, cible
GET /docs	Documentation Swagger générée	Consultation du schéma et essais

PredictionRequest contient site_id, crop et history. Le crop admis est wheat ; le site appartient aux trois identifiants configurés. history contient exactement trente objets WeatherDay, ordonnés et consécutifs. Chaque objet porte date et les sept variables météo. Les champs supplémentaires et valeurs non finies sont refusés. La configuration de sol est associée au site par le modèle et n’est pas librement fournie dans la requête.

Cette étape est illustrée à la figure 10.

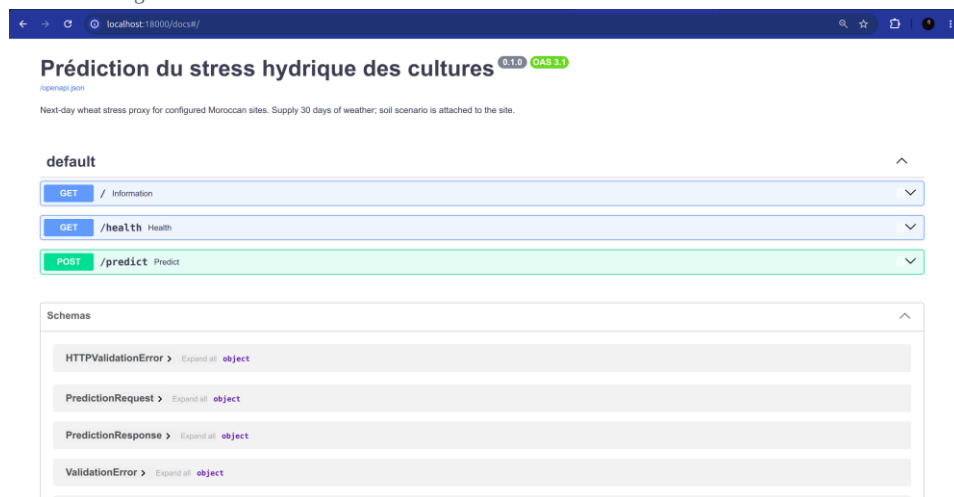


Figure 10 — Documentation Swagger des endpoints réels.

Source : Capture existante du dépôt ; les schémas sont définis dans src/api/schemas.py.

8.2 Erreurs, réponse et journalisation

Une requête invalide, une séquence temporelle incorrecte ou une prévision hors saison donne une réponse 422. Un modèle absent ou corrompu laisse l’application importable, mais la disponibilité et la prédiction répondent 503. Les erreurs de validation sont résumées sans recopier les corps bruts malformés. Les événements valides incluent la version du modèle, le site, la date, les features et un identifiant de requête.

L’exemple enregistré prédit 0,8685826307973173 pour le 16 novembre 2022, avec la version v4-2771746c21f7. Les seuils low < 0,3, medium < 0,6 et high sinon servent à l’interprétation visuelle ; ils ne définissent pas une décision d’irrigation validée.

Source : src/api/main.py ; src/api/schemas.py ; src/models/predict.py ; artifacts/example_request.json. Référence : [R4].

8.3 Streamlit, client du service de prédiction

Streamlit est un client HTTP de FastAPI. Il contrôle l'état de l'API, construit une requête complète, appelle POST /predict et affiche la réponse. Aucun estimateur ML n'est chargé dans l'interface pour reproduire la prédiction localement. Les deux modes utilisent le même composant de résultat et le même contrat.

Cette étape est illustrée à la figure 11.

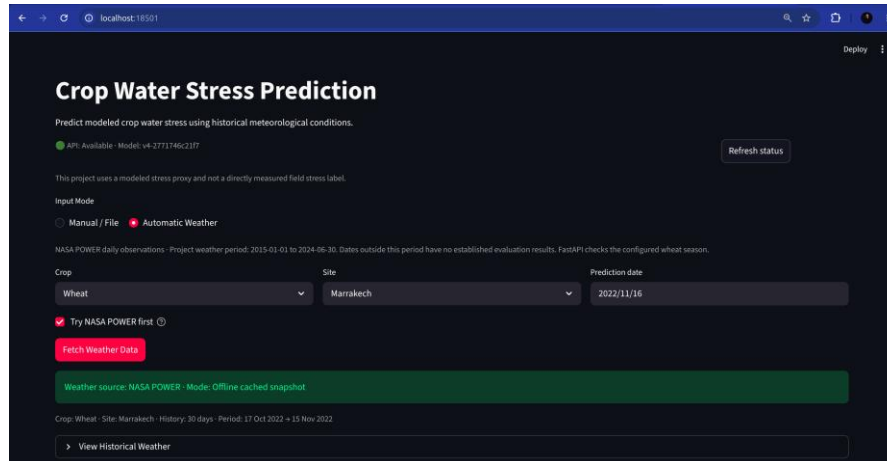


Figure 11 — Mode Automatic Weather avec instantané NASA vérifié en cache.

Source : Capture existante du dépôt : source et mode de récupération sont affichés à l'utilisateur.

Dans Manual / File, Load Example charge artifacts/example_request.json. L'utilisateur peut téléverser un JSON ou modifier le tableau des trente jours. Dans Automatic Weather, il choisit une culture, un site et une date cible ; l'interface construit la fenêtre qui précède cette date. L'exemple du 16 novembre 2022 emploie les observations du 17 octobre au 15 novembre, sans utiliser la météo de la date cible.

Cette étape est illustrée à la figure 12.

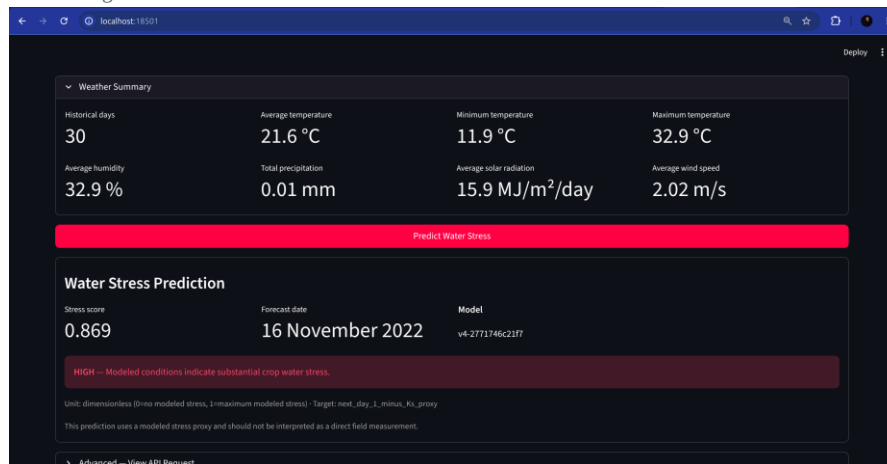


Figure 12 — Affichage de la prévision dans l'interface Streamlit.

Source : Capture existante ; le score arrondi 0,869 correspond à l'exemple de référence.

Les instantanés adaptés sont vérifiés avant réutilisation. L'option Try NASA POWER first permet une tentative de récupération, puis un repli vers un cache vérifié en cas d'échec. En l'absence de données valides, l'interface explique l'échec et propose le mode manuel ; elle ne fabrique pas d'observations. Un changement de sélection invalide le résultat précédent pour éviter de montrer une prévision associée à un ancien contexte.

Source : ui/app.py ; ui/utils.py ; ui/weather.py ; tests/test_weather_ui.py. Référence : [R12].

Chapitre 9 — Conteneurisation et déploiement

9.1 Services et réseau Docker

Le Dockerfile utilise une base python:3.12-slim et des cibles api, ui et tracking. Les services s'exécutent avec l'utilisateur UID 10001. Le modèle de production est inclus dans l'image API ; le build vérifie son intégrité et la prédiction de l'exemple. Le démarrage ne nécessite donc ni entraînement ni téléchargement NASA.

Tableau 17 — Services Compose, ports par défaut et persistance

Service	Port interne	Liaison hôte par défaut	Volume
api	8000	0.0.0.0:18000	api-events → /app/monitoring
ui	8501	0.0.0.0:18501	weather-cache → /app/data/raw/ui_weather
mlflow	5000	127.0.0.1:15000	mlflow-data → /mlflow

Cette étape est illustrée à la figure 13.

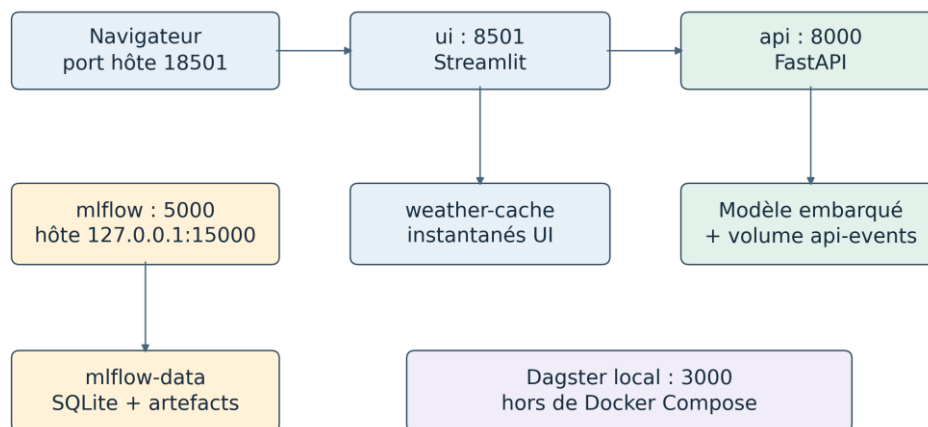


Figure 13 — Services Docker Compose et séparation du Dagster local.

Source : Schéma construit à partir du Dockerfile, de Compose et du Makefile.

L'interface appelle `http://api:8000` grâce au réseau Compose. Les ports publiés concernent l'accès depuis l'hôte et ne remplacent pas cette adresse interne. Le service ui attend une API saine ; MLflow est indépendant du serving. Les volumes conservent événements, cache et tracking après recréation des conteneurs, sous réserve de ne pas les supprimer.

9.2 Komodo et limites de preuve

Le README décrit une Stack Compose Komodo liée au dépôt Git, avec construction activée et récupération d'une image préconstruite désactivée. Les images sont construites à partir du dépôt. Aucun service Dagster n'est ajouté à Compose : `make dagster` lance un outil local sur le port 3000.

Les rapports du dépôt attestent une vérification Docker locale. Aucun accès au serveur Komodo distant ni capture de son interface n'a été trouvé dans les quatre captures disponibles. Le rapport décrit donc la configuration de déploiement et les vérifications locales, sans affirmer un état distant actuel. Les ports de Compose font autorité lorsque des notes historiques mentionnent d'anciens ports. MLflow n'étant pas authentifié par cette configuration, son accès hôte est limité à la boucle locale.

Source : Dockerfile ; `docker-compose.yml` ; README.md, Komodo Deployment ; `docs/deployment-verification.md`. Références : [R10, R13].

Chapitre 10 — Intégration continue

10.1 Workflows GitHub Actions

Deux workflows se déclenchent sur push, pull_request et workflow_dispatch. Ils installent Python 3.12 et disposent de permissions contents: read. La CI constitue une vérification reproductible des changements ; elle n'est pas une preuve de validité au champ du modèle.

Tableau 18 — Étapes réelles des workflows CI

Workflow	Étapes principales	Limite explicite
Python quality and tests	Installation verrouillée ; Ruff ; pytest ; vérification du modèle et des instantanés ; import API	Pas de construction Docker ni déploiement
Optional DataOps pipeline	Deux environnements ; tests DataOps ; démo Dagster jusqu'aux features ; contrôle de la release ; documentation dbt	Pas de collecte NASA en direct ni entraînement
Archivage DataOps	Téléversement ingestion.json, quality.json, manifest.json, run_results.json, catalog.json	Preuve de cette exécution, pas sauvegarde complète de l'application

La CI DataOps utilise les instantanés réels versionnés. Les tests d'intégration effectuent des chargements dlt et des transformations dbt dans des bases temporaires, puis injectent volontairement des anomalies pour vérifier leur rejet. Les fixtures de corruption ne remplacent pas les observations du corpus scientifique.

10.2 Résultats distants vérifiés

Tableau 19 — Exécutions GitHub Actions du commit final audité

Workflow	Identifiant d'exécution	Commit	État relevé
Python quality and tests	35234911963	0f3bc50	completed / success
Optional DataOps pipeline	35234911964	0f3bc50	completed / success

Ces résultats ont été lus dans l'API GitHub le 17 septembre 2026 ; leurs liens figurent en bibliographie [R14, R15]. Ils actualisent les anciennes notes du dépôt qui précisait que la publication n'avait pas encore eu lieu. Aucun résultat de Pull Request n'est attribué à ces exécutions : il s'agit de la vérification du commit publié.

10.3 Ce que la CI ne réalise pas

Aucune étape des workflows n'exécute docker compose build, ne publie une image dans un registre et ne déclenche une mise à jour Komodo. Les builds Docker cités dans les rapports sont des essais locaux distincts. Le projet implémente une intégration continue et une procédure de déploiement ; une livraison ou un déploiement continu entièrement automatisé n'est pas démontré. Cette distinction évite de présenter « CI/CD » comme un ensemble de fonctionnalités toutes actives.

Source : [.github/workflows/ci.yml](#) ; [.github/workflows/dataops.yml](#) ; [rapports/sources/verification_github.json](#). Référence : [R11].

Chapitre 11 — Monitoring et observabilité

11.1 Informations observées

Le monitoring est implémenté dans `monitoring/drift.py`. Il inspecte les variables explicatives et la distribution des prédictions. Pour chaque colonne, il recense les valeurs absentes ou non finies et celles qui dépassent les bornes disponibles. Une absence de colonne déclenche également une alerte qualité. Ces diagnostics complètent les validations de l'API ; ils permettent d'analyser des ensembles de données ou des événements collectés.

Tableau 20 — Indicateurs et seuils de monitoring configurés

Indicateur	Mise en œuvre	Seuil ou condition
Population Stability Index	Histogrammes sur quantiles de référence ; lissage des effectifs	Alerte si PSI > 0,20
Distance de Kolmogorov–Smirnov	Statistique <code>ks_2samp</code> sur valeurs valides	Alerte si distance > 0,15
Taille minimale	Effectifs référence et courant	Au moins 30 valeurs valides
Qualité	Colonnes absentes ; valeurs manquantes, non finies ou hors bornes	Alerte si un défaut est détecté
Performance	MAE, RMSE, R^2 si cible et prédiction sont disponibles	Comparaison après au moins 30 labels valides
Dégradation	Ratio MAE courante / MAE de référence	Alerte si ratio > 1,25

Le PSI compare des proportions par intervalles et la distance KS compare les distributions empiriques. Le code utilise la statistique KS, pas sa valeur p comme règle de décision. Les seuils sont des heuristiques de démonstration. L'autocorrélation, la saisonnalité et le mélange des sites peuvent expliquer des variations sans défaillance logicielle ni perte de qualité prédictive.

11.2 Événements de service et absence de labels

L'API écrit un journal JSONL comportant les événements de prédiction et les demandes invalides. Les événements valides contiennent un identifiant, la version du modèle, le site, la date cible, le score et les variables utilisées. Le lecteur de monitoring transforme ces événements en un jeu courant analysable.

Sans labels associés aux prédictions, la performance en production n'est pas calculable. Le programme indique alors `labels_unavailable`. Une dérive des entrées ou des prédictions n'est qu'un signal indirect. Pour mesurer une performance réelle, il faudrait collecter une cible cohérente, la joindre à chaque événement par site et date, puis tenir compte de la version du modèle et des différences de scénario.

Source : `monitoring/drift.py` ; `configs/config.yaml` ; `src/api/main.py` ; `tests/test_monitoring.py`.

11.3 Rapports historiques, simulés et événements réels

Les trois rapports conservés correspondent à des situations différentes. Ils ne doivent pas être présentés comme trois évaluations de production équivalentes. Les sorties JSON permettent une inspection des valeurs ; les sorties HTML offrent une lecture tabulaire des diagnostics.

Tableau 21 — Lecture des rapports de monitoring présents dans le dépôt

Rapport	Référence / courant	Qualité	Dérive	Performance
historical_report	537 / 1 074 lignes	Pas d'alerte	Alerte	Labels proxy disponibles ; ratio MAE 1,0886 ; pas d'alerte de dégradation
simulated_report	537 / 400 lignes	Alerte	Alerte	labels_unavailable
live_report	537 / 1 ligne	Pas d'alerte	Pas d'alerte globale	labels_unavailable ; une requête invalide comptabilisée

Dans le scénario historique, les observations de test sont comparées à une référence de validation. Les prédictions de référence précèdent le réajustement sur apprentissage et validation ; le ratio de performance est donc un indicateur heuristique, pas une expérience contrôlée de dégradation du même modèle dans le temps.

Le scénario simulé déplace délibérément certaines variables : températures augmentées de 6 °C, humidité réduite, cumul de pluie modifié, puis valeurs manquantes et précipitations négatives injectées. Le code signale explicitement qu'il ne s'agit pas d'une météo future physiquement cohérente. L'alerte démontre la capacité des contrôles à réagir à une perturbation construite ; elle ne prouve pas une dérive observée sur une exploitation agricole.

Le rapport live ne contient qu'une prédiction. Son drapeau global `drift_alert=false` ne permet pas de conclure à l'absence de dérive : les effectifs sont inférieurs au minimum de trente et les colonnes sont marquées `insufficient_data`. La distinction entre « pas de signal calculable » et « stabilité démontrée » est essentielle pour interpréter correctement le tableau.

11.4 Observabilité opérationnelle

Dagster fournit une observabilité des étapes de données ; MLflow décrit les expériences ; les healthchecks contrôlent la disponibilité des services ; les événements API documentent l'inférence. Ces mécanismes répondent à des questions différentes. Aucun serveur Prometheus, tableau Grafana, envoi d'alerte par messagerie ou réentraînement automatique sur dérive n'est présent. Une extension future devra définir les destinataires, la fréquence d'analyse et la politique de traitement des alertes.

Source : `monitoring/historical_report.json` ; `monitoring/simulated_report.json` ; `monitoring/live_report.json` ; `monitoring/drift.py`.

Chapitre 12 — Résultats, limites et discussion

12.1 Bilan scientifique et opérationnel

Le modèle retenu atteint une RMSE de 0,188591 et un R^2 de 0,625080 sur 1 074 observations de test. Ces résultats établissent une capacité à approximer le proxy dans le périmètre étudié. La persistance conserve une erreur nettement plus faible. L'apport principal du projet réside donc aussi dans la construction d'un cycle de vie complet, traçable et démontrable.

Le chemin DataOps reproduit exactement le CSV météo et le CSV de features du chemin initial : les empreintes correspondantes sont identiques dans les preuves conservées. Cette équivalence est plus forte qu'une simple comparaison du nombre de lignes. Les vérifications locales documentées couvrent dlt, dbt, les contrats, les jobs Dagster, le registre MLflow, les endpoints et les interactions Streamlit. Les deux workflows distants du commit audité sont également réussis.

Tableau 22 — Limites, conséquences et pistes d'amélioration

Limite vérifiée	Conséquence	Travail futur proposé
Proxy et sols hypothétiques	Pas de validation du stress réel ni de prescription d'irrigation	Collecter des mesures de terrain et calibrer le bilan
Trois sites connus, blé uniquement	Généralisation spatiale et culturelle inconnue	Validation par site exclu et autres cultures
Fenêtre de 30 jours	Mémoire limitée face à l'état hydrique complet	Comparaison à information égale et analyse de la mémoire utile
Test déjà observé	Risque de sélection indirecte lors d'itérations futures	Définir une nouvelle réserve temporelle
Pas d'incertitude prédictive	Score ponctuel sans intervalle	Évaluation par blocs saisonniers et méthodes d'incertitude
DuckDB local	Concurrence d'écriture limitée	Sérialiser les runs ; élargir l'architecture seulement si nécessaire
Pas de labels live	Dérive indirecte, performance non mesurable	Jointure audité avec des observations de référence
PR et cérémonies non établies	Collaboration formelle partiellement démontrée	Réaliser des revues effectives et documenter les réunions
Déploiement distant non audité	Pas de preuve de disponibilité actuelle du serveur	Vérification distante et conservation de preuves datées

12.2 Priorités raisonnables

La priorité scientifique est l'acquisition de labels observés et l'analyse des hypothèses de sol, avant l'augmentation de la complexité du modèle. La priorité opérationnelle est une meilleure documentation des revues, des sauvegardes et des incidents. L'ajout de services distribués ne serait justifié que par des contraintes réelles de volume ou de disponibilité, absentes de cette démonstration locale.

Source : `artifacts/evaluation/v4-2771746c21f7/test_metrics.json` ; `docs/dataops-evidence.json` ; code et configuration audités.

Conclusion générale

Ce projet met en œuvre une chaîne DataOps/MLOps de bout en bout pour la prévision à un jour d'un indicateur de stress hydrique modélisé du blé. Les observations NASA POWER sont conservées dans des instantanés vérifiables, normalisées, contrôlées et transformées en variables causales. La cible est construite par un bilan hydrique explicite, dont les hypothèses et les limites restent visibles.

La couche DataOps ajoute une ingestion réelle avec dlt, un stockage analytique DuckDB, des transformations dbt assorties de contrats et une orchestration Dagster. Son caractère parallèle protège le pipeline initial et rend possible une comparaison exacte des sorties. L'équivalence des données canoniques et des features permet de présenter les outils comme des composants intégrés, et non comme des démonstrations indépendantes sans lien avec le problème ML.

La chaîne MLOps organise la comparaison chronologique, l'enregistrement des expériences, le versionnement des candidats, l'évaluation figée et la promotion contrôlée. FastAPI expose un contrat d'inférence précis ; Streamlit fournit une interface HTTP commune aux modes manuel et automatique. Docker Compose reproduit les services applicatifs, tandis que GitHub Actions exécute les contrôles de code et de données à partir d'instantanés locaux.

Le monitoring distingue qualité des entrées, dérive des distributions et performance lorsque des labels existent. Cette distinction évite d'interpréter une alerte statistique comme une perte de précision démontrée, ou un manque d'échantillons comme une stabilité acquise. La traçabilité des artefacts et des prédictions facilite l'analyse, mais ne remplace pas des observations au champ.

Les résultats doivent être lus avec la même rigueur. Le modèle sélectionné est meilleur que les autres configurations ML évaluées sur la validation, mais la persistance de l'état simulé obtient une meilleure performance finale. La différence d'information disponible rend cette comparaison instructive sans constituer une démonstration de supériorité du ML. Le système n'est pas présenté comme une solution d'irrigation certifiée.

Les perspectives prioritaires concernent la calibration des hypothèses, l'acquisition de mesures indépendantes, la validation sur de nouveaux sites et saisons, et l'estimation d'incertitude. Sur le plan collectif, de véritables revues de Pull Requests et des traces de cérémonies Agile complèteraient les preuves techniques existantes. Le résultat obtenu constitue ainsi une base cohérente pour poursuivre simultanément la validation scientifique et l'amélioration de l'exploitation logicielle.

Bibliographie et sources

Les références externes ci-dessous servent à documenter les outils et la méthode. Les faits d'implémentation, résultats et figures sont principalement issus du dépôt audité au commit 0f3bc50. Les documentations en ligne évoluent ; les versions installées sont précisées dans les fichiers requirements. Consultation : 17 septembre 2026.

[R1] NASA POWER. Documentation de l'API quotidienne. <https://power.larc.nasa.gov/docs/services/api/temporal/daily/>

[R2] Allen, R. G., Pereira, L. S., Raes, D. et Smith, M. (1998). Crop evapotranspiration — Guidelines for computing crop water requirements. FAO Irrigation and Drainage Paper 56, Rome. <https://www.fao.org/4/X0490E/x0490e00.htm>

[R3] MLflow. Documentation officielle : suivi d'expériences et registre des modèles. Version du projet : 3.1.1. <https://mlflow.org/docs/latest/ml/model-registry/workflow/>

[R4] FastAPI. Documentation officielle. Version du projet : 0.115.14. <https://fastapi.tiangolo.com/>

[R5] dlt. Destination DuckDB et pipelines de chargement. Version du projet : 1.30.0. <https://dlthub.com/docs/dlt-ecosystem/destinations/duckdb>

[R6] DuckDB. Documentation officielle. Version du projet : 1.3.2. <https://duckdb.org/docs/stable/>

[R7] dbt Labs. Model contracts. dbt-core 1.10.11 ; adaptateur dbt-duckdb 1.9.6. <https://docs.getdbt.com/docs/mesh/govern/model-contracts>

[R8] Dagster. Asset jobs. Version du projet : 1.11.11. <https://docs.dagster.io/guides/build/jobs/asset-jobs>

[R9] scikit-learn. Ensemble methods et gradient boosting. Version du projet : 1.7.0. <https://scikit-learn.org/stable/modules/ensemble.html>

[R10] Docker. Documentation Docker Compose. <https://docs.docker.com/compose/>

[R11] GitHub. Documentation GitHub Actions. <https://docs.github.com/en/actions>

[R12] Streamlit. Documentation officielle. Version du projet : 1.55.0. <https://docs.streamlit.io/>

[R13] Komodo. Déploiement de stacks Docker Compose. <https://komo.do/docs/deploy/compose>

[R14] Dépôt Anass-Erf/MLOPS_Project. Exécution Python quality and tests du commit 0f3bc50. https://github.com/Anass-Erf/MLOPS_Project/actions/runs/35234911963

[R15] Dépôt Anass-Erf/MLOPS_Project. Exécution Optional DataOps pipeline du même commit. https://github.com/Anass-Erf/MLOPS_Project/actions/runs/35234911964

Sources primaires internes : README.md ; docs/methodology.md ; src/ ; configs/ ; tests/ ; dbt_project/ ; artifacts/ ; monitoring/ ; .github/workflows/ ; historique Git. Les légendes indiquent le fichier d'origine des figures et les fins de section précisent les principales preuves techniques.

Annexes

Annexe A — Architecture et frontières opérationnelles

Le chemin historique conserve les étapes collect_data → preprocess → build_features → train → evaluate → registry. La commande make pipeline enchaîne ces modules et inclut la promotion. Le chemin DataOps exécute dlt → DuckDB → dbt → contrat → features, puis permet un entraînement et une évaluation explicites dans un espace distinct. Aucune de ces chaînes n'est déclenchée par le lancement des services applicatifs.

Tableau 23 — Emplacements et responsabilités du dépôt

Répertoire ou fichier	Contenu et rôle
configs/	Données, scénarios, splits, modèles et instance Dagster
src/data/	Collecte NASA, normalisation et validation
src/dataops/	Chargement dlt, contrat, dbt, orchestration et pont ML
src/features/	Variables causales et simulateur du proxy
src/models/	Comparaison, évaluation, registre et prédiction
src/api/ ; ui/	Service FastAPI et client Streamlit
dbt_project/	Modèles SQL, contrats et tests
data/raw/	Instantanés JSON et manifestes immuables
artifacts/	Métriques, figures, exemples et provenance
artifacts/dataops/	Base et résultats générés du chemin optionnel, ignorés par Git
models/	Pointeurs et versions locales, dont la release servie
monitoring/	Analyse de dérive, rapports et événements
tests/ ; tests/dataops/	Tests unitaires et intégration optionnelle
.github/workflows/	Deux workflows d'intégration continue

Annexe B — Arborescence fonctionnelle simplifiée

```

project/
  configs/          data/raw/
  src/
    data/           dataops/
    features/       models/
    api/            utils/
  dbt_project/
    models/staging/ models/marts/
    tests/
  ui/               monitoring/
  tests/            notebooks/
  artifacts/        models/
  docs/             screenshots/
  Dockerfile        docker-compose.yml
  Makefile          requirements*.txt

```

Cette vue résume les fichiers effectivement présents. Les environnements Python, bases locales et états d'exécution ne sont pas confondus avec les sources versionnées. Les quatre captures disponibles concernent Streamlit, Swagger et MLflow ; aucune capture Komodo n'est incluse.

Annexe C — Contrat des endpoints FastAPI

L'appel POST /predict reçoit un objet contenant site_id, crop et history. Un WeatherDay comporte date, temperature, temperature_max, temperature_min, precipitation, humidity, wind_speed et solar_radiation. L'ordre chronologique et les trente jours consécutifs sont obligatoires. L'API produit les champs suivants :

Tableau 24 — Champs de la réponse de prédiction

Champ	Type ou valeurs	Exemple enregistré
prediction	Réel borné dans [0 ; 1]	0,8685826307973173
model_version	Chaîne	v4-2771746c21f7
forecast_date	Date	2022-11-16
unit	Chaîne descriptive	Indice sans dimension
stress_level	low, medium, high	high
target	Identifiant de la cible	next_day_1_minus_Ks_proxy

Les erreurs de schéma et de saison répondent 422 ; un modèle indisponible répond 503. Les routes GET / et GET /health permettent respectivement de consulter le périmètre et l'état du modèle. Swagger est accessible par GET /docs.

Annexe D — Commandes de démonstration

Tableau 25 — Commandes du Makefile et effets à connaître

Commande	Effet
make offline	Vérifier/réutiliser les instantanés NASA locaux
make preprocess ; make features	Construire le jeu canonique puis les variables et la cible
make train ; make evaluate	Produire un candidat puis l'évaluer
make promote	Modifier explicitement la production après contrôles
make api ; make ui ; make mlflow	Démarrer séparément les services locaux
make test lint	Vérifications initiales et qualité Python
make dataops-install	Installer l'environnement optionnel séparé
make dataops-ingest	Charger les données réelles dans DuckDB
make dataops-dbt ; make dataops-quality	Construire/tester les tables puis approuver l'export
make dataops-inspect	Afficher les relations, dates et sources
make dataops-demo ; make dagster	Exécuter la démo sans entraînement puis lancer l'interface
make dataops-train ; make dataops-evaluate	Modélisation explicite dans le stockage de démonstration
make dataops-test ; make dataops-dbt-docs	Tests optionnels et génération du lineage SQL

Les commandes avec plusieurs étapes sont à exécuter séquentiellement. Pour la démonstration courte, installer les environnements à l'avance, montrer un manifeste, ingérer, inspecter, tester, puis afficher le graphe et les matérialisations. La présence d'une commande d'entraînement ne signifie pas qu'elle doit être lancée au démarrage.

Annexe E — Synthèse du Data Contract

Tableau 26 — Contrat structurel, temporel et de provenance

Dimension	Règle exacte ou comportement
Schéma canonique	13 colonnes dans l'ordre attendu par src/dataops/contract.py
Types	Date pandas datetime ; valeurs météo numériques et finies
Valeurs manquantes	Refus d'un jeu vide ou de données météo obligatoires manquantes
Unicité	Une observation par couple site_id/date
Continuité	Jours ordonnés et consécutifs par site
Couverture DataOps	Tous les jours configurés, tous les sites configurés, aucun site inconnu
Scénario	Coordonnées et paramètres de sol égaux aux valeurs configurées
Source	Endpoint NASA et paramètres de requête concordants
Intégrité	SHA-256 du fichier brut égal au manifeste
Équivalence	Valeurs canoniques exactement égales à la normalisation indépendante
Publication	Remplacement atomique de weather.csv et stockage de son empreinte
Consommation ML	Empreinte approuvée et empreinte météo du manifeste de features concordantes

Annexe F — Tests et modèles dbt

Les 35 tests de données se répartissent entre trente contrôles `not_null`, deux contrôles `accepted_values` sur `site_id` et trois requêtes SQL singulières. Les contrats de modèles s'ajoutent à ces tests. `stg_weather` déclare dix-sept colonnes, dont quatre de provenance ; `ml_weather` en expose treize.

```
-- Extrait fidèle : détection des doublons site/date
select site_id, date, count(*) as observations
from {{ ref('ml_weather') }}
group by site_id, date
having count(*) <> 1
```

`weather_rules.sql` couvre les bornes météo et la cohérence des températures et du sol. `complete_dates.sql` construit le calendrier attendu pour chaque site et le compare aux données par jointure externe complète. Cette stratégie détecte également un site entièrement absent, ce qu'un simple test de continuité à l'intérieur des groupes présents ne suffirait pas à garantir.

Les tests d'intégration créent une base temporaire, effectuent deux ingestions, exécutent dbt puis injectent des anomalies. Ils vérifient que les tests échouent et que le CSV approuvé précédent n'est pas remplacé par des données invalides. Cette vérification négative renforce la portée de la réussite nominale.

Annexe G — Assets et jobs Dagster

Tableau 27 — Dépendances et effets des assets

Asset	Dépendance	Effet
nasa_snapshots	Source externe	Représentation des instantanés existants
ingest_weather	nasa_snapshots	Chargement dlt hors ligne dans DuckDB
dbt_transform	ingest_weather	dbt build : tables et tests
data_quality	dbt_transform	Vérification Python, équivalence et CSV approuvé
build_features	data_quality	Appel du feature builder initial dans l'espace DataOps
train_model	build_features	Entraînement volontaire ; candidat MLflow isolé
evaluate_model	train_model	Évaluation figée du candidat, sans promotion

Les quatre premiers assets exécutables constituent dataops_demo. Les deux derniers constituent train_evaluate_explicit. Le lien de dépendance rend le chemin lisible, mais le job d'entraînement séparé suppose une préparation réussie des features. Le contrôle d'empreinte empêche l'usage d'un export modifié ou de features obsolètes.

Annexe H — Docker Compose

```
# Extrait fidèle de docker-compose.yml
ui:
  build:
    context: .
    dockerfile: Dockerfile
    target: ui
  environment:
    CWS_API_URL: http://api:8000
  ports:
    - "${CWS_BIND_ADDRESS:-0.0.0.0}:${UI_HOST_PORT:-18501}:8501"
  volumes:
    - weather-cache:/app/data/raw/ui_weather
  depends_on:
    api:
      condition: service_healthy
  restart: unless-stopped
```

Les deux autres services sont api et mlflow. Le premier expose l'inférence et persiste ses événements ; le second stocke SQLite et les artefacts dans mlflow-data. Les trois images sont construites depuis le même Dockerfile multi-cibles. La configuration n'intègre pas Dagster et ne télécharge pas NASA au lancement. Les secrets d'une plateforme externe ne sont pas nécessaires au scénario local.

Annexe I — Intégration continue et vérifications

Le workflow initial fixe Python 3.12, installe requirements.txt, exécute Ruff puis pytest, contrôle les artefacts de déploiement et importe l'API. Le workflow DataOps ajoute l'installation des deux environnements et les commandes ci-dessous. L'extrait est limité aux étapes d'exécution, sans les en-têtes du fichier.

```
- run: make install dataops-install
- run: make dataops-test
- name: Materialize offline Dagster demo through existing feature engineering
  run: make dataops-demo
- name: Verify existing deployment assets and generate dbt lineage
  run: |
    .venv/bin/python scripts/verify_deployment_assets.py
    make dataops-dbt-docs
```

L'étape upload-artifact conserve des éléments de preuve même en cas d'échec, grâce à if: always(). L'analyse d'un run doit cependant consulter ses statuts effectifs : la présence d'un fichier archivé ne démontre pas à elle seule la réussite de toutes les étapes.

Tableau 28 — Matrice finale de conformité aux exigences du module

Exigence	Constat issu de l'audit
Backlog et User Stories	Formalisés dans ce rapport à partir des besoins implémentés
Trois sprints, Review, Retrospective	Reconstruction pédagogique explicitement signalée
dlt, DuckDB, dbt, Dagster	Implémentations réelles présentes et preuves d'exécution conservées
Qualité, contrat et lineage	Validations Python/SQL, contrats, manifestes et graphes
ML, évaluation, versionnement	Quatre modèles, partitions chronologiques, MLflow et artefacts figés
Monitoring	Qualité, dérive et performance conditionnée aux labels
Git et GitHub	Historique et branche publiés ; deux workflows réussis sur 0f3bc50
Pull Requests	Aucune PR renvoyée par l'API consultée ; procédure seule disponible
Docker / FastAPI / Streamlit	Services, contrôles et captures disponibles
Komodo	Procédure documentée ; état distant actuel non vérifié
CD automatique	Non implémenté dans les workflows audités

Annexe J — Exemple JSON de référence

Les deux pages suivantes reproduisent les trente journées de artifacts/example_request.json. Le code est compacté pour la mise en page, sans modification des valeurs. Les blocs doivent être réunis pour former l'objet complet ; aucun jour de la fenêtre n'est remplacé par des points de suspension. La réponse attendue est celle exposée au chapitre 8. Le fichier original reste la source directement utilisable avec make demo-request.

Annexe J — Requête JSON complète

Reproduction du fichier de référence ; concaténer les deux blocs sans leurs titres.

```
{
  "site_id": "marrakech",
  "crop": "wheat",
  "history": [
    {"date": "2022-10-17", "humidity": 39.76, "precipitation": 0.01, "solar_radiation": 16.4,
      "temperature": 24.7, "temperature_max": 32.59, "temperature_min": 17.8, "wind_speed": 2.52},
    {"date": "2022-10-18", "humidity": 37.41, "precipitation": 0.0, "solar_radiation": 15.24,
      "temperature": 25.06, "temperature_max": 32.46, "temperature_min": 20.25, "wind_speed": 2.02},
    {"date": "2022-10-19", "humidity": 37.42, "precipitation": 0.0, "solar_radiation": 17.58,
      "temperature": 24.43, "temperature_max": 32.87, "temperature_min": 18.3, "wind_speed": 1.91},
    {"date": "2022-10-20", "humidity": 39.7, "precipitation": 0.0, "solar_radiation": 17.33,
      "temperature": 23.77, "temperature_max": 32.48, "temperature_min": 17.34, "wind_speed": 2.4},
    {"date": "2022-10-21", "humidity": 37.16, "precipitation": 0.0, "solar_radiation": 18.69,
      "temperature": 23.55, "temperature_max": 32.57, "temperature_min": 17.04, "wind_speed": 2.28},
    {"date": "2022-10-22", "humidity": 33.63, "precipitation": 0.0, "solar_radiation": 18.22,
      "temperature": 23.15, "temperature_max": 31.88, "temperature_min": 16.72, "wind_speed": 2.4},
    {"date": "2022-10-23", "humidity": 31.38, "precipitation": 0.0, "solar_radiation": 17.01,
      "temperature": 22.18, "temperature_max": 31.37, "temperature_min": 15.59, "wind_speed": 2.28},
    {"date": "2022-10-24", "humidity": 26.81, "precipitation": 0.0, "solar_radiation": 16.48,
      "temperature": 23.17, "temperature_max": 31.88, "temperature_min": 16.23, "wind_speed": 2.57},
    {"date": "2022-10-25", "humidity": 28.97, "precipitation": 0.0, "solar_radiation": 14.85,
      "temperature": 23.51, "temperature_max": 31.84, "temperature_min": 17.69, "wind_speed": 2.66},
    {"date": "2022-10-26", "humidity": 26.44, "precipitation": 0.0, "solar_radiation": 16.98,
      "temperature": 24.5, "temperature_max": 32.52, "temperature_min": 18.63, "wind_speed": 2.18},
    {"date": "2022-10-27", "humidity": 26.51, "precipitation": 0.0, "solar_radiation": 18.38,
      "temperature": 23.08, "temperature_max": 32.2, "temperature_min": 16.0, "wind_speed": 1.72},
    {"date": "2022-10-28", "humidity": 23.42, "precipitation": 0.0, "solar_radiation": 17.8,
      "temperature": 22.48, "temperature_max": 31.6, "temperature_min": 14.91, "wind_speed": 1.76},
    {"date": "2022-10-29", "humidity": 24.51, "precipitation": 0.0, "solar_radiation": 13.97,
      "temperature": 22.91, "temperature_max": 30.31, "temperature_min": 16.86, "wind_speed": 2.18},
    {"date": "2022-10-30", "humidity": 27.48, "precipitation": 0.0, "solar_radiation": 15.69,
      "temperature": 22.51, "temperature_max": 30.61, "temperature_min": 17.56, "wind_speed": 1.49},
    {"date": "2022-10-31", "humidity": 27.65, "precipitation": 0.0, "solar_radiation": 17.39,
      "temperature": 21.93, "temperature_max": 29.78, "temperature_min": 15.8, "wind_speed": 1.99},
  ]
}
```

Annexe J — Requête JSON complète (suite)

Reproduction du fichier de référence ; concaténer les deux blocs sans leurs titres.

```
{
  "date": "2022-11-01", "humidity": 34.79, "precipitation": 0.0, "solar_radiation": 12.01,
  "temperature": 21.46, "temperature_max": 29.87, "temperature_min": 17.08, "wind_speed": 1.97},
  {"date": "2022-11-02", "humidity": 36.17, "precipitation": 0.0, "solar_radiation": 14.69,
  "temperature": 20.85, "temperature_max": 28.81, "temperature_min": 15.09, "wind_speed": 2.21},
  {"date": "2022-11-03", "humidity": 39.99, "precipitation": 0.0, "solar_radiation": 17.04,
  "temperature": 19.8, "temperature_max": 26.92, "temperature_min": 15.3, "wind_speed": 1.71},
  {"date": "2022-11-04", "humidity": 40.86, "precipitation": 0.0, "solar_radiation": 16.67,
  "temperature": 19.84, "temperature_max": 27.22, "temperature_min": 15.42, "wind_speed": 1.66},
  {"date": "2022-11-05", "humidity": 38.96, "precipitation": 0.0, "solar_radiation": 16.81,
  "temperature": 20.42, "temperature_max": 28.69, "temperature_min": 14.59, "wind_speed": 1.35},
  {"date": "2022-11-06", "humidity": 42.68, "precipitation": 0.0, "solar_radiation": 16.79,
  "temperature": 20.64, "temperature_max": 29.3, "temperature_min": 15.17, "wind_speed": 1.63},
  {"date": "2022-11-07", "humidity": 32.77, "precipitation": 0.0, "solar_radiation": 16.63,
  "temperature": 20.73, "temperature_max": 29.45, "temperature_min": 14.34, "wind_speed": 2.07},
  {"date": "2022-11-08", "humidity": 30.47, "precipitation": 0.0, "solar_radiation": 12.92,
  "temperature": 20.04, "temperature_max": 27.82, "temperature_min": 15.18, "wind_speed": 1.83},
  {"date": "2022-11-09", "humidity": 35.8, "precipitation": 0.0, "solar_radiation": 13.78,
  "temperature": 18.14, "temperature_max": 25.3, "temperature_min": 13.24, "wind_speed": 1.56},
  {"date": "2022-11-10", "humidity": 41.15, "precipitation": 0.0, "solar_radiation": 16.22,
  "temperature": 18.18, "temperature_max": 26.44, "temperature_min": 13.2, "wind_speed": 1.67},
  {"date": "2022-11-11", "humidity": 29.57, "precipitation": 0.0, "solar_radiation": 16.31,
  "temperature": 18.5, "temperature_max": 27.64, "temperature_min": 11.93, "wind_speed": 2.07},
  {"date": "2022-11-12", "humidity": 25.74, "precipitation": 0.0, "solar_radiation": 15.45,
  "temperature": 19.87, "temperature_max": 28.68, "temperature_min": 13.07, "wind_speed": 3.32},
  {"date": "2022-11-13", "humidity": 26.58, "precipitation": 0.0, "solar_radiation": 12.89,
  "temperature": 19.74, "temperature_max": 27.77, "temperature_min": 13.63, "wind_speed": 2.01},
  {"date": "2022-11-14", "humidity": 30.05, "precipitation": 0.0, "solar_radiation": 14.03,
  "temperature": 20.0, "temperature_max": 27.46, "temperature_min": 14.13, "wind_speed": 1.18},
  {"date": "2022-11-15", "humidity": 32.64, "precipitation": 0.0, "solar_radiation": 13.66,
  "temperature": 19.08, "temperature_max": 26.97, "temperature_min": 13.53, "wind_speed": 1.91}
}
```